UNIVERSITAS
GADJAH MADA

Perancangan *Soft Sensor* Deteksi Aktivitas Manusia Menggunakan Data *Gyroscope* dan Akselerometer dari Perangkat Mobile

Menggunakan Metode Random Forest dan KNN

Kelompok 11

M. Nadhir Al Ghifary (22/503570/TK/55030)

Natasya Aisyah Dini (22/503833/TK/55079)

Melathi Andhara Putri (22/503582/TK/55033)

Ahmad Fauzan Bakri (22/497813/TK/54577)

LOCALLY ROOTED,

GLOBALLY RESPECTED

ugm.ac.id

Deskripsi Dataset

Human Activity Recognition Dataset

Deskripsi Dataset

UNIVERSITAS GADJAH MADA

Dataset KU-HAR adalah kumpulan data **Human Activity Recognition (HAR)** yang berisi informasi dari 90 partisipan yang menggunakan sensor akselerometer dan gyroscope pada smartphone. Dataset ini terdiri dari **1945 data aktivitas mentah** dan **9185 subsample** yang diperoleh hasil ekstraksi. Dengan **sampling rate 100 Hz** dan tanpa proses filtering, data ini mencatat berbagai aktivitas manusia, mulai dari gerakan sederhana seperti **berdiri diam, berjalan**, hingga gerakan yang lebih kompleks seperti **naik tangga** dan **bermain tenis meja**.

Jumlah Aktivitas

Terdapat 12 aktivitas dalam dataset ini, namun hanya 5 aktivitas yang dipilih:

- **Stand** → (1 min)
- **Sit** → (1 min)
- **Lay** → (1 min)
- **Walk** → (≈12 s)
- **Run** → (≈7 s)

Masalah yang Dapat Diselesaikan

1. Pemantauan aktivitas fisik secara real-time tanpa perangkat tambahan.
2. Deteksi dini perubahan pola aktivitas yang dapat mengindikasikan gangguan kesehatan.
3. Pemantauan aktivitas lansia atau pasien secara non-invasif dalam konteks perawatan jarak jauh.
4. Identifikasi jenis aktivitas pengguna (seperti berjalan, duduk, berlari) hanya melalui smartphone.

Visualisasi Data

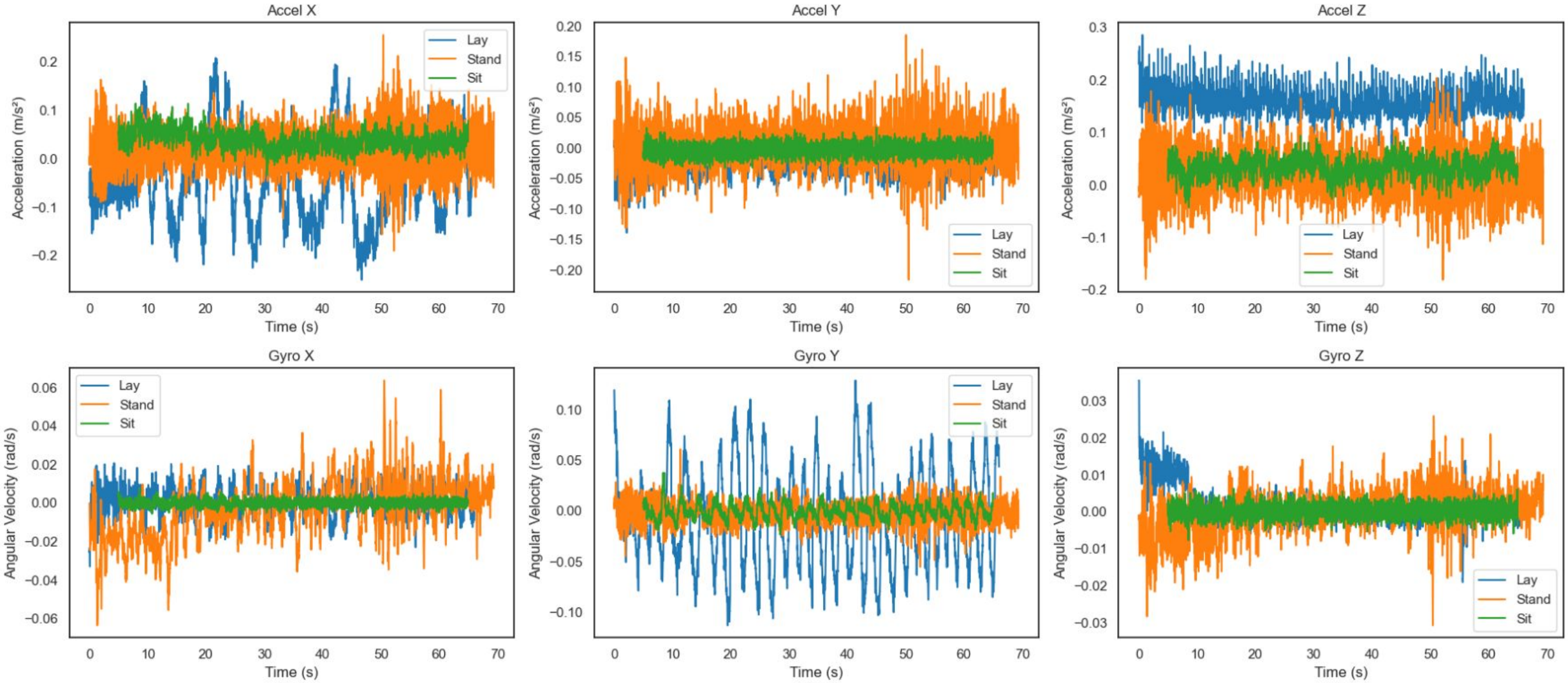
```python
set_theme(style='white', palette=None)
color_pal = plt.rcParams['axes.prop_cycle'].by_key()['color']
color_cycle = cycle(plt.rcParams['axes.prop_cycle'].by_key()['color'])

# Buat figure dan axes
fig, axes = plt.subplots(2, 3, figsize=(18, 8))  # 2 baris, 3 kolom

def tabel (axe, fitur, judul, satuan):
    axe.plot(df_lay['time'], df_lay[fitur], label='Lay')
    axe.plot(df_stand['time'], df_stand[fitur], label='Stand')
    axe.plot(df_sit['time'], df_sit[fitur], label='Sit')
    axe.set_title(judul)
    axe.set_xlabel('Time (s)')
    axe.set_ylabel(satuan)
    axe.legend()

tabel (axes[0, 0], 'accel_x', 'Accel X', 'Acceleration (m/s²)')
tabel (axes[0, 1], 'accel_y', 'Accel Y', 'Acceleration (m/s²)')
tabel (axes[0, 2], 'accel_z', 'Accel Z', 'Acceleration (m/s²)')
tabel (axes[1, 0], 'gyro_x', 'Gyro X', 'Angular Velocity (rad/s)')
tabel (axes[1, 1], 'gyro_y', 'Gyro Y', 'Angular Velocity (rad/s)')
tabel (axes[1, 2], 'gyro_z', 'Gyro Z', 'Angular Velocity (rad/s)')

# Atur layout biar rapi
plt.tight_layout()

# Tampilkan plot
plt.show()
```

UNIVERSITAS GADJAH MADA

Visualisasi data 60 sekon Stand, Sit, Lay

Tampilan hasil visualisasi data

UNIVERSITAS GADJAH MADA

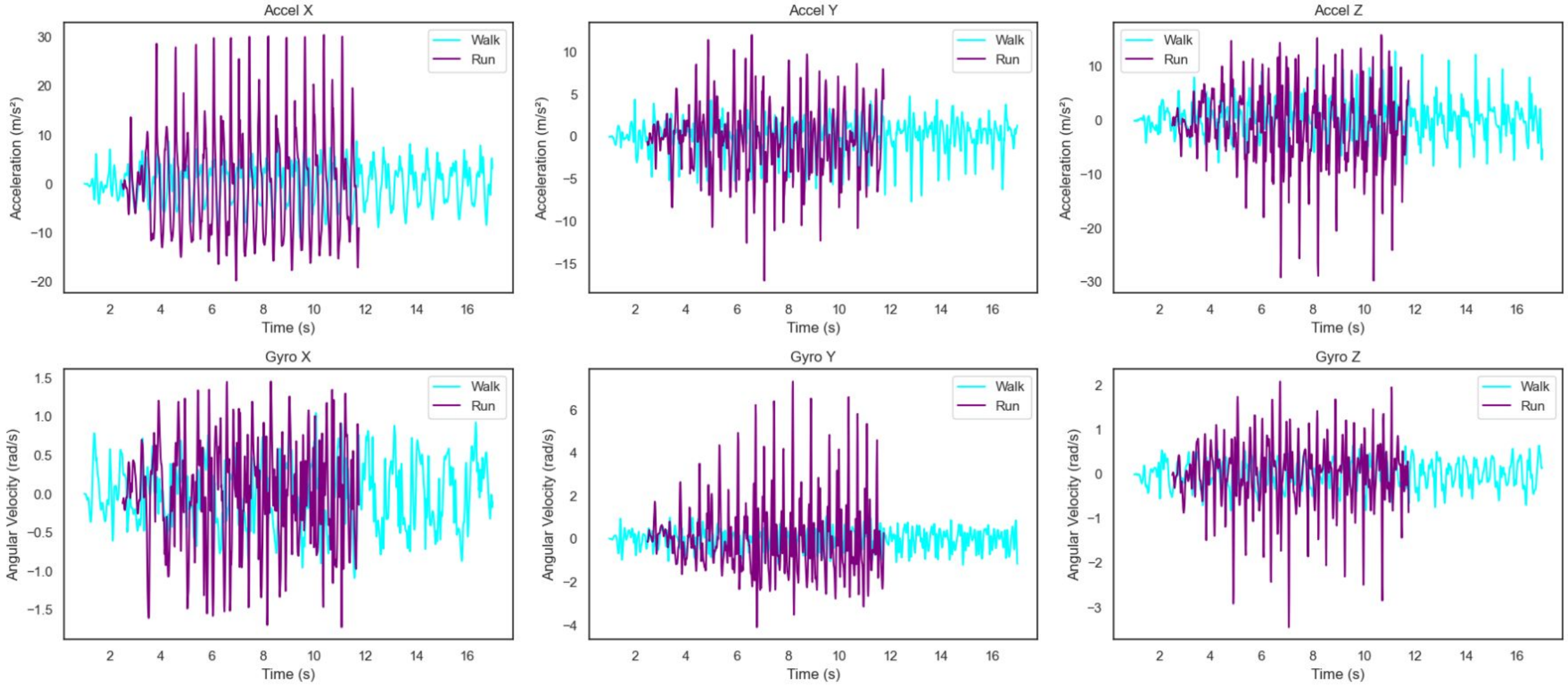


```python
# Buat figure dan axes
fig, axes = plt.subplots(2, 3, figsize=(18, 8))  # 2 baris, 3 kolom

def tabel (axe, fitur, judul, satuan):
    axe.plot(df_walk['time'], df_walk[fitur], label='Walk', color="cyan")
    axe.plot(df_run['time'], df_run[fitur], label='Run', color="purple")
    axe.set_title(judul)
    axe.set_xlabel('Time (s)')
    axe.set_ylabel(satuan)
    axe.legend()

tabel (axes[0, 0], 'accel_x', 'Accel X', 'Acceleration (m/s²)')
tabel (axes[0, 1], 'accel_y', 'Accel Y', 'Acceleration (m/s²)')
tabel (axes[0, 2], 'accel_z', 'Accel Z', 'Acceleration (m/s²)')
tabel (axes[1, 0], 'gyro_x', 'Gyro X', 'Angular Velocity (rad/s)')
tabel (axes[1, 1], 'gyro_y', 'Gyro Y', 'Angular Velocity (rad/s)')
tabel (axes[1, 2], 'gyro_z', 'Gyro Z', 'Angular Velocity (rad/s)')

# Atur layout biar rapi
plt.tight_layout()

# Tampilkan plot
plt.show()
```


Visualisasi data 12 sekon Run and Walk

Tampilan hasil visualisasi data

UNIVERSITAS GADJAH MADA



Feature Extraction

Feature Extraction

Feature Extraction diperlukan karena prediksi aktivitas **tidak dapat dilakukan hanya berdasarkan data sesaat**. Data sensor mengalami fluktuasi dalam rentang waktu tertentu, sehingga dibutuhkan proses ekstraksi fitur untuk **menangkap pola perubahan** tersebut.

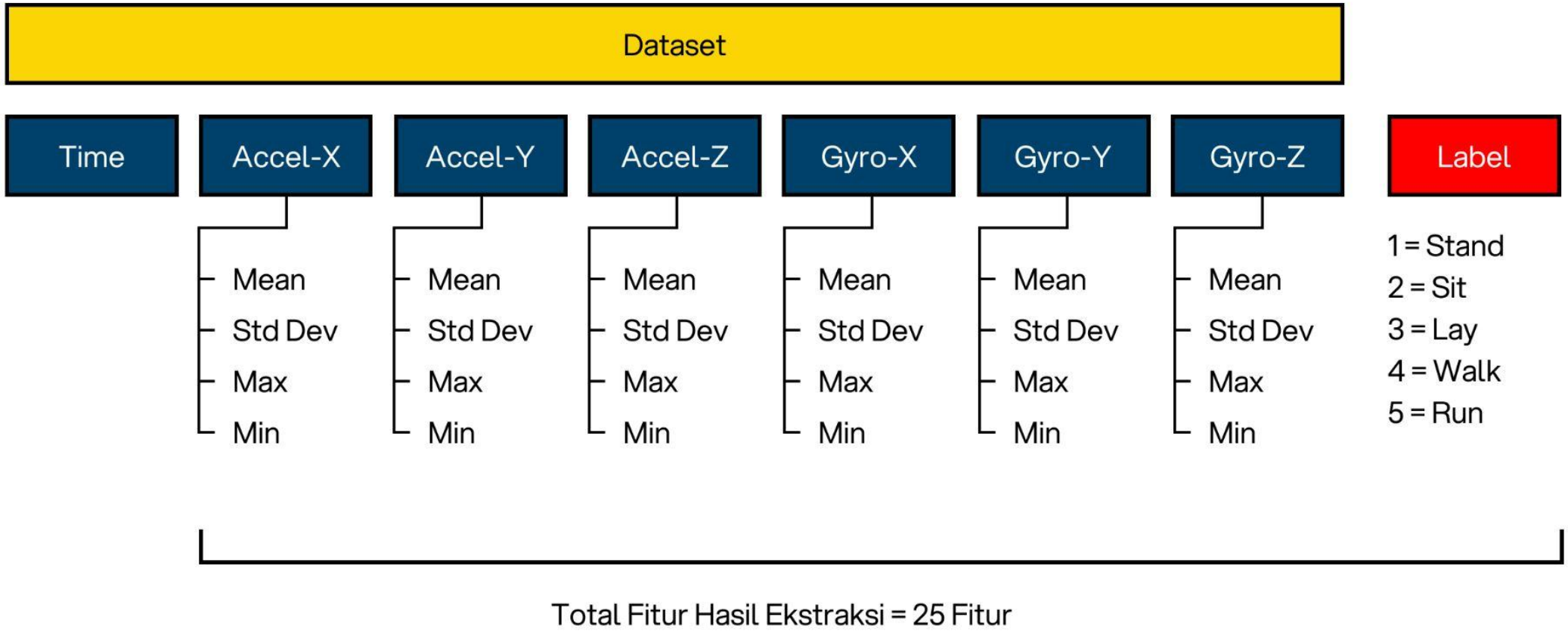
Feature extraction dilakukan dengan membagi data ke dalam window waktu tertentu.

- Aktivitas stand, lay, dan sit, proses ekstraksi dilakukan per 3 sekon (mengingat total durasi data: 60 sekon)
- aktivitas run dan walk, ekstraksi dilakukan per 1 sekon (mengingat total durasi data: 12 sekon)

Dalam setiap window waktu tersebut, diambil beberapa nilai statistik utama sebagai fitur, yaitu:

- Mean
- Standard Deviation
- Maximum Value
- Minimum Value

Skema Feature Extraction



Hasil
Feature Extraction

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 9856 entries, 0 to 9855
Data columns (total 25 columns):
 #   Column        Non-Null Count  Dtype
---  ------        --------------  -----
 0   mean_accel_x  9856 non-null   float64
 1   std_accel_x   9854 non-null   float64
 2   max_accel_x   9856 non-null   float64
 3   min_accel_x   9856 non-null   float64
 4   mean_accel_y  9856 non-null   float64
 5   std_accel_y   9854 non-null   float64
 6   max_accel_y   9856 non-null   float64
 7   min_accel_y   9856 non-null   float64
 8   mean_accel_z  9856 non-null   float64
 9   std_accel_z   9854 non-null   float64
 10  max_accel_z   9856 non-null   float64
 11  min_accel_z   9856 non-null   float64
 12  mean_gyro_x   9856 non-null   float64
 13  std_gyro_x    9854 non-null   float64
 14  max_gyro_x    9856 non-null   float64
 15  min_gyro_x    9856 non-null   float64
 16  mean_gyro_y   9856 non-null   float64
 17  std_gyro_y    9854 non-null   float64
 18  max_gyro_y    9856 non-null   float64
 19  min_gyro_y    9856 non-null   float64
 20  mean_gyro_z   9856 non-null   float64
 21  std_gyro_z    9854 non-null   float64
 22  max_gyro_z    9856 non-null   float64
 23  min_gyro_z    9856 non-null   float64
 24  label         9856 non-null   int64
```

Terdapat data jumlah data yang tidak seragam karena **data hilang**

Oleh karena itu, diperlukan **data cleaning**

Data Cleaning

Import Dataframe Hasil Ekstraksi

UNIVERSITAS GADJAH MADA

```python
# Path folder tempat CSV hasil ekstraksi
output_dir = os.getcwd()

# Path masing-masing file CSV
stand_csv = os.path.join(output_dir, "stand_features.csv")
sit_csv   = os.path.join(output_dir, "sit_features.csv")
lay_csv   = os.path.join(output_dir, "lay_features.csv")
walk_csv  = os.path.join(output_dir, "walk_features.csv")
run_csv   = os.path.join(output_dir, "run_features.csv")

# Load ketiga CSV jadi DataFrame
df_stand = pd.read_csv(stand_csv)
df_sit   = pd.read_csv(sit_csv)
df_lay   = pd.read_csv(lay_csv)
df_walk  = pd.read_csv(walk_csv)
df_run   = pd.read_csv(run_csv)

# Gabungkan ketiganya
df = pd.concat([df_stand, df_sit, df_lay, df_walk, df_run], ignore_index=True)
```

Pengecekan Data Hilang

UNIVERSITAS GADJAH MADA

```python
# Cek jumlah NaN di tiap fitur
nan_count = df.isna().sum()

# Tampilkan hasil
print("Jumlah NaN di tiap fitur:")
print(nan_count)
```

```
Jumlah NaN di tiap fitur:
mean_accel_x    0
std_accel_x     2
max_accel_x     0
min_accel_x     0
mean_accel_y    0
std_accel_y     2
max_accel_y     0
min_accel_y     0
mean_accel_z    0
std_accel_z     2
max_accel_z     0
min_accel_z     0
mean_gyro_x     0
std_gyro_x      2
max_gyro_x      0
min_gyro_x      0
mean_gyro_y     0
std_gyro_y      2
max_gyro_y      0
min_gyro_y      0
mean_gyro_z     0
std_gyro_z      2
max_gyro_z      0
min_gyro_z      0
label           0
dtype: int64
```

Terdapat data hilang untuk beberapa fitur sehingga perlu dihilangkan

Menghilangkan NaN

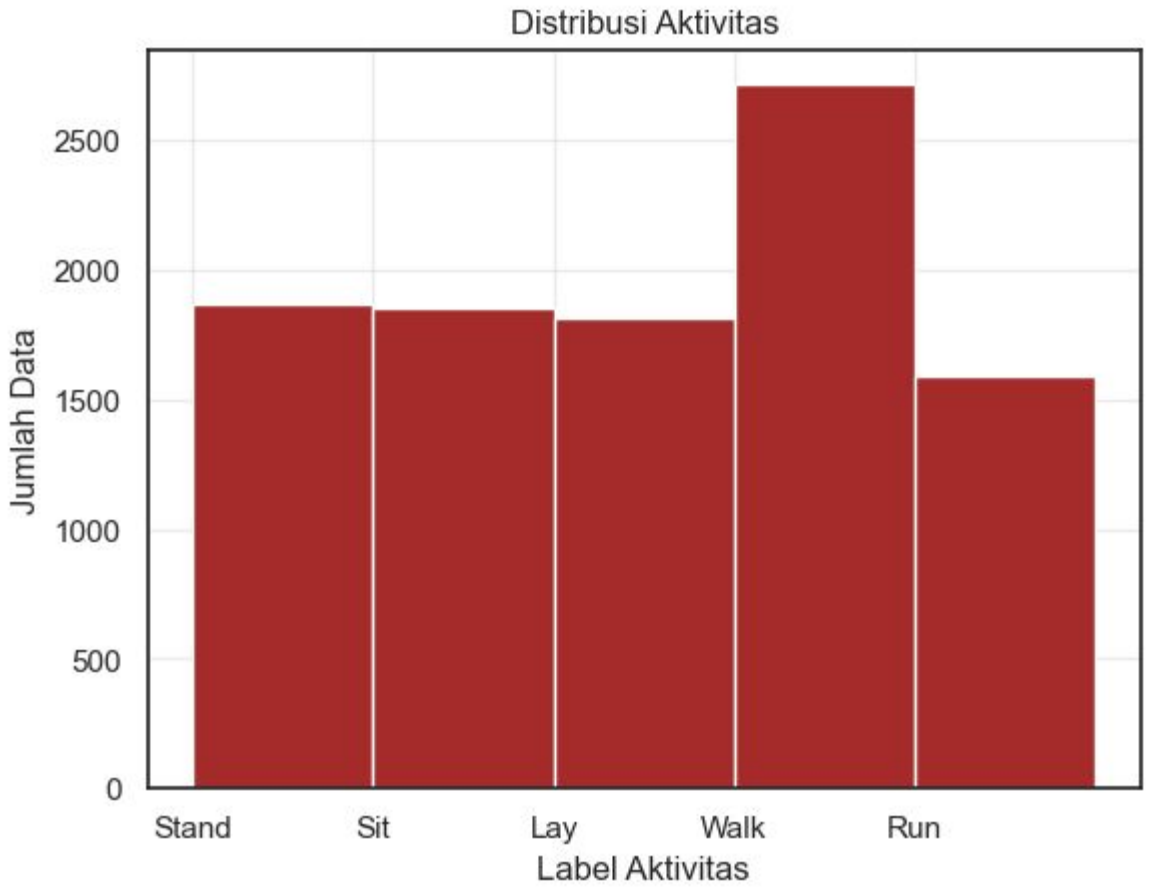
UNIVERSITAS GADJAH MADA

```python
# Drop baris yang mengandung NaN
df_clean = df.dropna()

# Cek kembali setelah di-drop
nan_count_after = df_clean.isna().sum().reset_index()
nan_count_after.columns = ['Fitur', 'Jumlah NaN']

print("\nJumlah NaN setelah di-drop:")
print(nan_count_after)

# Tampilkan jumlah data awal dan setelah di-drop
print(f"\nJumlah data sebelum di-drop: {len(df)}")
print(f"Jumlah data setelah di-drop : {len(df_clean)}")
```

```
Jumlah NaN setelah di-drop:
           Fitur  Jumlah NaN
0   mean_accel_x           0
1    std_accel_x           0
2    max_accel_x           0
3    min_accel_x           0
4   mean_accel_y           0
5    std_accel_y           0
6    max_accel_y           0
7    min_accel_y           0
8   mean_accel_z           0
9    std_accel_z           0
10   max_accel_z           0
11   min_accel_z           0
12   mean_gyro_x           0
13    std_gyro_x           0
14    max_gyro_x           0
15    min_gyro_x           0
16   mean_gyro_y           0
17    std_gyro_y           0
18    max_gyro_y           0
19    min_gyro_y           0
20   mean_gyro_z           0
21    std_gyro_z           0
...
24         label           0

Jumlah data sebelum di-drop: 9856
Jumlah data setelah di-drop : 9854
```

Analisis Statistik

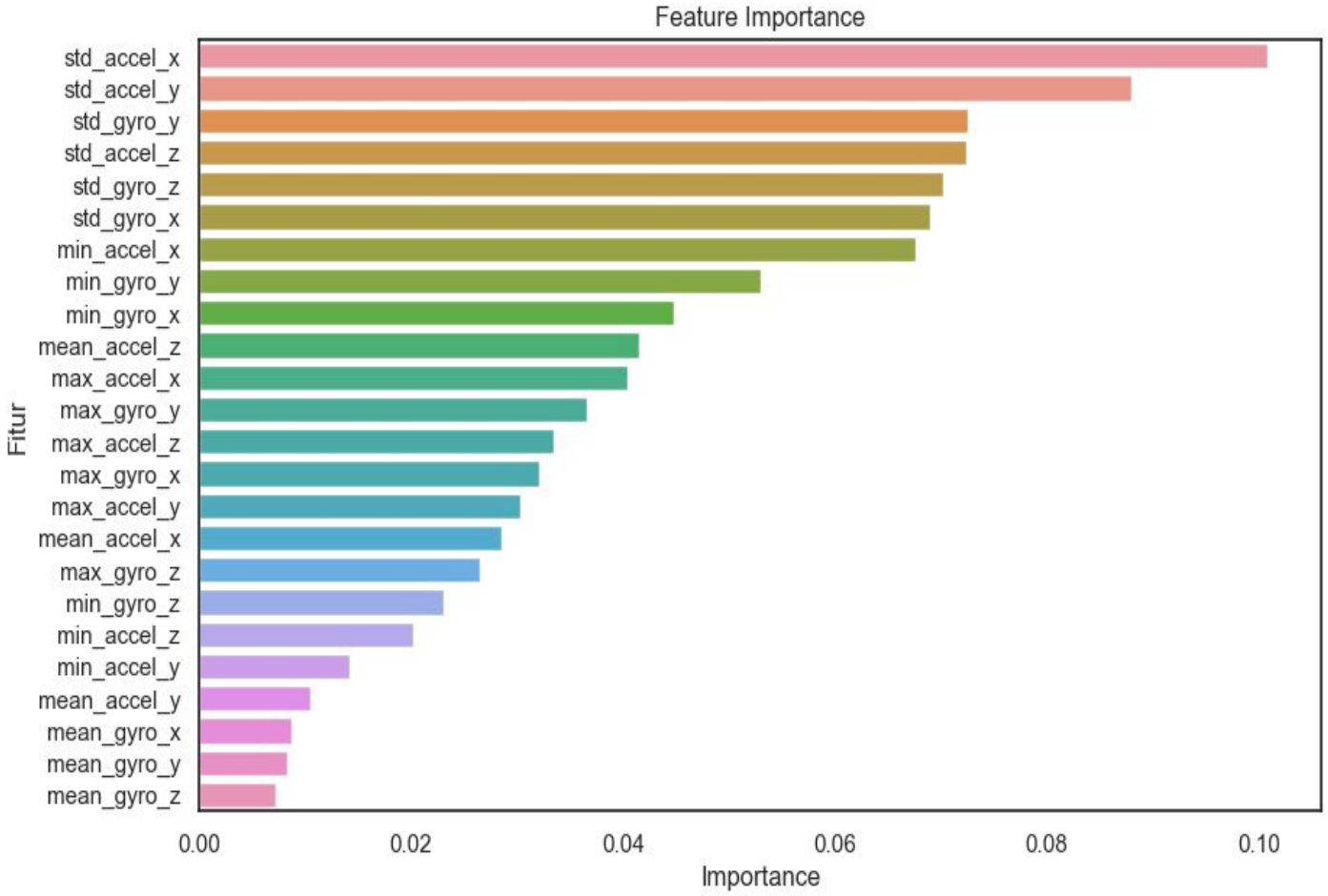
Analisis Distribusi Tiap Aktivitas



Interpretasi Grafik:

- Walk (berjalan) memiliki jumlah data tertinggi, sekitar 2500 entri.
- Stand, Sit, dan Lay memiliki distribusi jumlah data yang cukup seimbang, dengan sekitar 1700-2000 entri.
- Run (berlari) sedikit lebih rendah dibandingkan dengan Walk, namun tetap memiliki jumlah data yang signifikan (sekitar 2000 entri).
- Data sudah cukup banyak untuk setiap aktivitas sehingga sudah cukup untuk dimodelkan

Pengaruh Variabel Input Terhadap Variabel Output



- **Sumbu X:** Menunjukkan tingkat pentingnya setiap fitur.
- **Sumbu Y:** Menunjukkan fitur-fitur yang digunakan untuk mendeteksi aktivitas.

Pengaruh Fitur Terhadap Prediksi Aktivitas

Fitur dengan Importance Tertinggi:

- **std_accel_x**, **std_accel_y**, dan **std_accel_z** (standar deviasi akselerasi pada sumbu X, Y, dan Z) memiliki kontribusi terbesar dalam prediksi aktivitas.

Fitur dengan Importance Terendah:

- **mean_gyro_y** dan **mean_gyro_z** (rata-rata gyroskop pada sumbu Y dan Z) memiliki kontribusi lebih kecil.

Variabilitas akselerasi pada sumbu X sangat penting dalam membedakan aktivitas. Fluktuasi akselerasi (misalnya, berjalan atau berlari) lebih berpengaruh daripada nilai rata-rata atau ekstrem.

Feature Engineering

Feature Engineering

UNIVERSITAS GADJAH MADA

Dalam hal ini digunakan metode **Train, Test, Split** yang disediakan oleh Scikit Learn untuk memisahkan data training, data test, dan data validasi. Adapun persentase dan jumlah data tiap klasifikasi data dapat dilihat pada tabel berikut

Data Training	**Data Test**	**Data Validasi**
6306 (64%)	1971 (20%)	1577 (16%)

Model Building

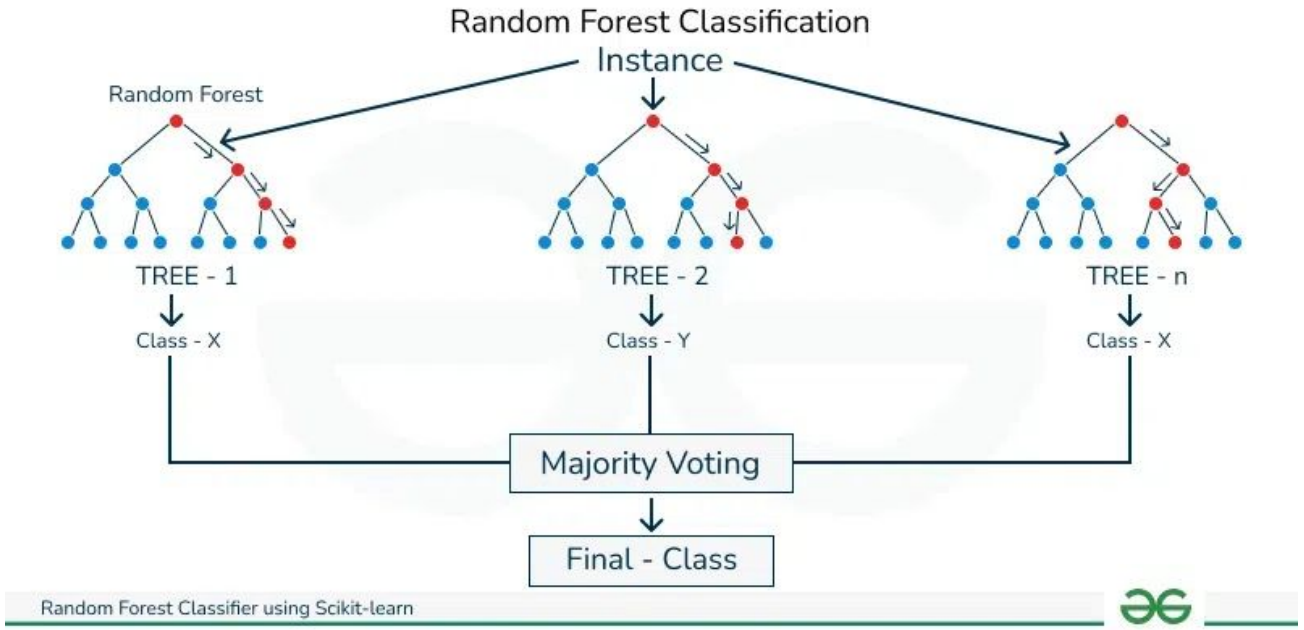
Random Forest Classifier

Overview Model Random Forest Classifier

Random Forest adalah salah satu algoritma machine learning yang membangun banyak decision tree dan menggabungkan hasilnya untuk menghasilkan prediksi

Pada prosesnya:

- Data akan dibagi secara acak untuk membangun beberapa decision tree.
- Setiap tree memberikan hasil prediksi.
- Hasil akhir ditentukan berdasarkan mayoritas voting (untuk klasifikasi)



Training Model Random Forest Classifier

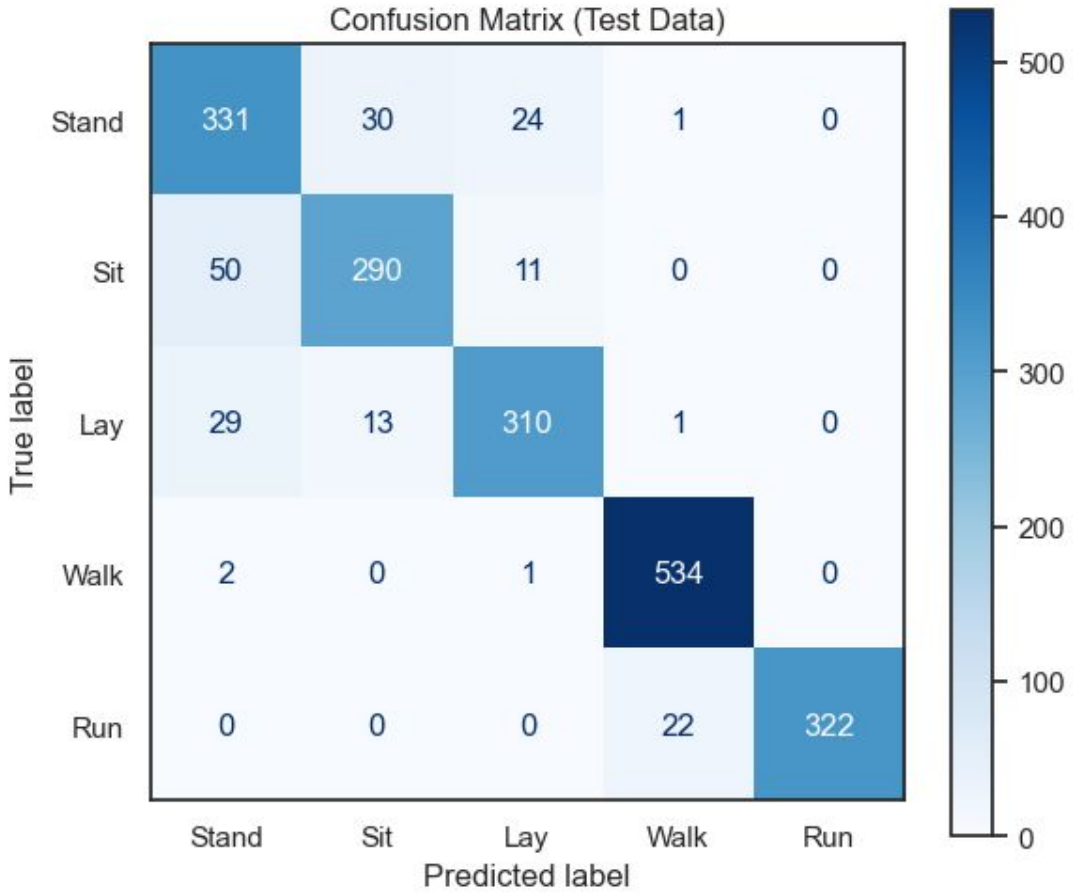
UNIVERSITAS GADJAH MADA

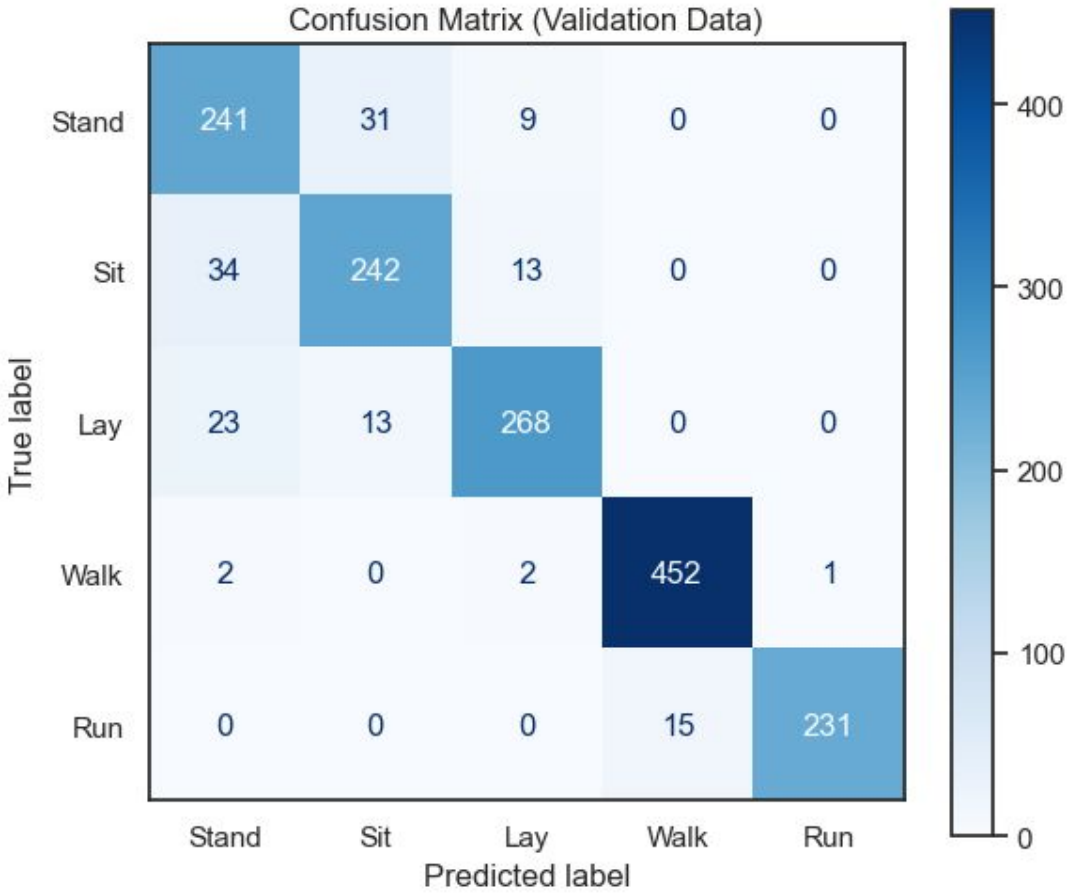
Parameter	Default Value	Keterangan
`n_estimators`	`100`	Jumlah decision tree yang akan dibuat dalam random forest.
`max_depth`	`None`	Batas kedalaman tiap decision tree. Jika `None`, pohon akan dibuat hingga daun.
`min_samples_split`	`2`	Jumlah minimal sampel untuk membagi node.
`min_samples_leaf`	`1`	Jumlah minimal sampel yang harus ada di leaf node.
`random_state`	`42`	Seed untuk pengacakan, supaya hasil bisa direproduksi.
`n_jobs`	`None`	Jumlah core CPU yang dipakai. Jika `-1`, gunakan semua core.

Hasil Model Random Forest Classifier

Parameter	**Random Forest Classifier**	
	Data Test	**Data Validasi**
Akurasi	0.954845	0.961319
Presisi	0.955405	0.962494
Recall	0.954845	0.961319
F1-Score	0.954917	0.961499

Confusion Matrix
Model Random Forest Classifier





Model Building

K-Neighbours Classifier

Overview Model K-Neighbours Classifier

UNIVERSITAS GADJAH MADA

Apa itu KNN?

- K-Nearest Neighbors (KNN) adalah algoritma klasifikasi berbasis *instance (lazy learner)*.
- Tidak mempelajari fungsi dari data saat training, melainkan menyimpan semua data dan membuat prediksi hanya saat ada data baru (*lazy*).
- Klasifikasi dilakukan dengan melihat k tetangga terdekat dari data baru dan melakukan voting mayoritas berdasarkan label tetangga tersebut.

Bagaimana Cara Kerjanya?

1. Hitung jarak (umumnya Euclidean) antara data baru dengan seluruh *data training*.
2. Ambil *k* data dengan jarak terpendek.
3. Prediksi kelas berdasarkan label mayoritas dari tetangga tersebut.

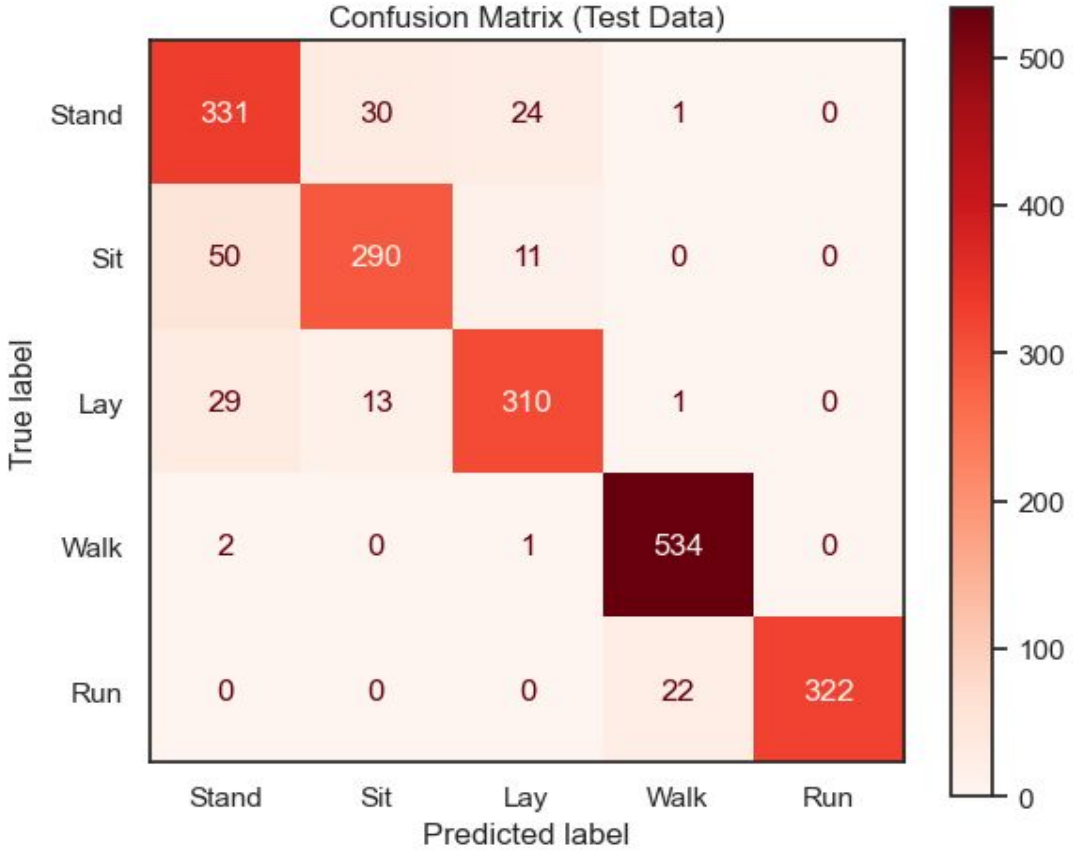
Hasil Model K-Neighbours Classifier

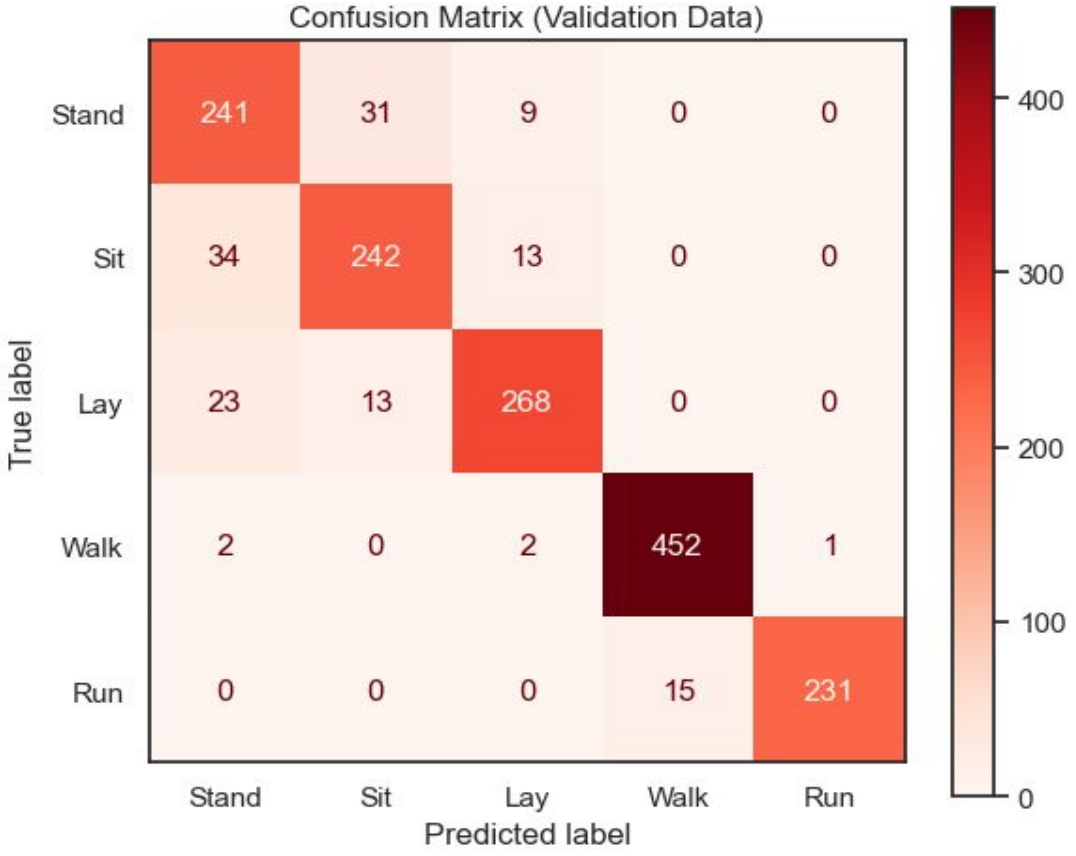
UNIVERSITAS GADJAH MADA

Parameter: k = 5, metrik jarak = Euclidean

Parameter	**Random Forest**	
	Data Test	**Data Validasi**
Akurasi	0.906646	0.909321
Presisi	0.908149	0.910938
Recall	0.906646	0.909321
F1-Score	0.906822	0.909734

Confusion Matrix
Model K-Neighbours Classifier





Kesimpulan

Kesimpulan

Parame-ter	Random Forest		K-Neighbour Classifier	
	Data Test	Data Validation	Data Test	Data Validation
Akurasi	0.954845	0.961319	0.906646	0.909321
Presisi	0.955405	0.962494	0.908149	0.910938
Recall	0.954845	0.961319	0.906646	0.909321
F1-Score	0.954917	0.961499	0.906822	0.909734

Model Random Forest memberikan hasil yang lebih baik dibandingkan KNN, dengan akurasi mencapai 95% pada deteksi aktivitas manusia, sedangkan KNN mencapai akurasi 90-91%. Dengan akurasi yang tinggi ini, sistem ini dapat di-deploy ke perangkat mobile sebagai soft sensor untuk pemantauan aktivitas fisik secara real-time.

Video Demo

https://youtu.be/rMV_IjNqGzE

Video Presentasi

https://youtu.be/Bgw_GsXl5WM

UNIVERSITAS
GADJAH MADA

Terima Kasih